

BrokerVerse OOTB

Insurance broking platform by iorta TechNXT

Solution Architecture · Document 01 of 11

Solution Component Diagram and Application Architecture

Product	BrokerVerse OOTB
Document	01 Solution Component Diagram and Application Architecture
Version	1.1
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse OOTB Solution Architecture: 01 Solution Component Diagram and Application Architecture
Document ID	BV-SA-01
Version	1.1
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, deploy/, docker-compose.yml, .github/workflows, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue
1.1	29 September 2026	iorta TechNXT	Updated to the current system: placement journey, co-insurance, posting rules, period end and BIR tax, bank reconciliation, document numbering, commission rate matrix, single PDF engine, uploads and go-live imports, housekeeping, business time zone, broker roles, deployment on the BrokerVerse URL

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
1.2 Architecture at a glance.....	4
2 Solution component diagram.....	5
2.1 Logical components.....	6
2.2 Front-end application structure.....	7
3 Deployment view.....	9
3.1 Production deployment on the existing BrokerVerse URL.....	9
3.2 Docker Compose.....	11
4 Request processing.....	13
4.1 API request pipeline.....	13
4.2 Authentication and session flow.....	15
5 Module map.....	17
5.1 Backend modules.....	17
5.2 Front-end areas and backend modules.....	19
6 Integration points.....	20
7 Security architecture summary.....	21
7.1 Roles and permissions.....	21
7.2 Controls.....	21
8 Key business flows.....	23
8.1 Lead, placement, policy and billing.....	23
8.2 Receipt, commission and remittance.....	24
8.3 Direct bill.....	25
8.4 Claims.....	26
8.5 Renewal.....	27
8.6 Period end.....	27
8.7 Scheduled jobs.....	27
9 Architecture decisions and constraints.....	29
10 Related documents.....	30
11 Glossary.....	31

1 Introduction

1.1 Purpose and scope

This document describes the application architecture of BrokerVerse OOTB, the insurance broking platform of iorta TechNXT for brokers in the Philippines. It covers the logical components, the deployment, how a request is processed and secured, how the functional modules map to the front end and the backend, the external integrations and the main business flows. It is the entry point to the Solution Architecture set. Documents 02 to 11 give the detail on the database, the technology, the shared services and operations.

The description is taken from the code and configuration of the baseline named in the document control section: the React front end in `brokerverse/`, the Node.js API in `backend/`, `docker-compose.yml`, the deployment package in `deploy/` (`README.md` and `REFERENCE.md`) and the GitHub workflows `.github/workflows/ci.yml` and `brokerverse/.github/workflows/deploy.yml`.

1.2 Architecture at a glance

Aspect	BrokerVerse OOTB
Style	Three-tier web application: single-page application (SPA), stateless REST API and a relational database. The server is a modular monolith.
Presentation tier	React 18 SPA built with Create React App and craco, PrimeReact components and Redux Toolkit. Served as static files from S3 and CloudFront on the existing BrokerVerse URL (nginx in the Docker Compose option).
Application tier	Node.js 22 (ES modules) with Express 4. 38 module folders mounted under <code>/api</code> with 717 registered routes, plus two health endpoints. A built-in scheduler runs 15 jobs in the business time zone (Asia/Manila).
Data tier	PostgreSQL 16 with the database time zone set to Asia/Manila: 138 tables from 58 migration files, applied on start under an advisory lock. Files on a persistent volume (<code>UPLOAD_DIR</code>).
Integration	Office 365 SMTP for outbound e-mail through an outbox table; public quote approval link for clients; spreadsheet uploads and go-live imports with 39 published templates; OpenAPI, Postman and Excel API documentation generated from the route registry.
Security	JWT access tokens (30 minutes) with rotating refresh tokens (30 days), token version revocation, role and permission checks on every route, role inheritance, maker-checker approvals, audit log, TOTP two-factor sign-in, HMAC-signed file links.
Deployment	AWS ap-southeast-1 on the existing BrokerVerse URL: the front end through its GitHub workflow to S3 and CloudFront; the backend container from <code>backend/Dockerfile</code> behind HTTPS; PostgreSQL 16; a persistent upload volume; secrets in a secret store. Docker Compose serves a single server or a test system.

2 Solution component diagram

2.1 Logical components

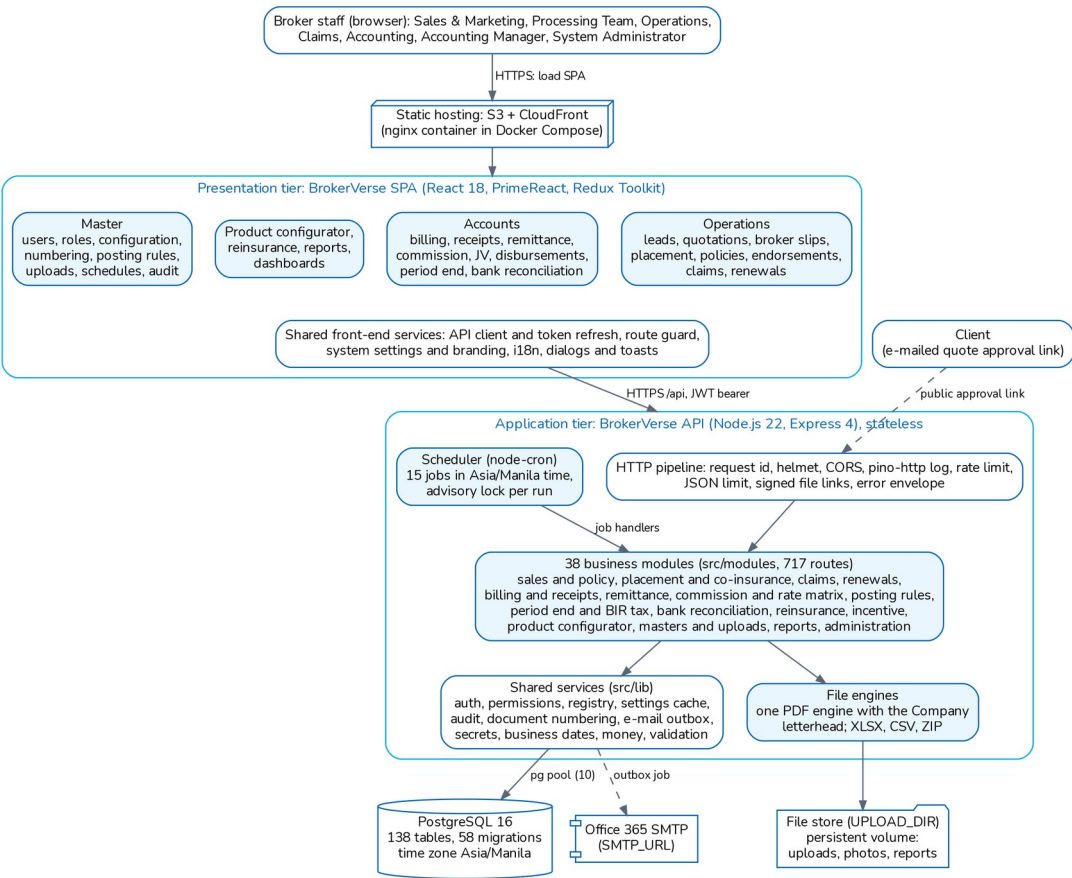


Figure 1. Logical component view of BrokerVerse OOTB

Component	Responsibility
Browser SPA	All user screens: operations (leads to renewals, including broker slips and placement slips), accounts (billing, receipts, remittance, commission, journals, period end, bank reconciliation, tax), product configurator, reinsurance, reports, dashboards, masters and administration. It holds the session tokens, renews them automatically and hides the menus a role may not open.
Static hosting	Serves the compiled SPA (HTML, JavaScript, CSS, images). There is no server-side rendering; every data call goes to <code>/api</code> .
HTTP pipeline	Request handling in <code>backend/src/app.js</code> : request id, security headers (helmet), CORS, structured access log with redaction, global rate limit, JSON size limit, signing of file URLs in responses, one error envelope.
Business modules	38 folders in <code>backend/src/modules</code> . Each has a <code>router.js</code> that declares its endpoints through the route registry, and services that hold the business rules and SQL. They are loaded automatically at start. The larger modules (accounting, placement, period-end, bank-reconciliation, remittance, reports, document-numbering) have their own README.
Shared services	<code>backend/src/lib</code> : authentication, permissions, settings cache, audit, document numbering, e-mail outbox, secrets and signed links, business dates, money rounding, validation, rate limits, health, the PDF engine and the letterhead (document 05).
Scheduler	<code>backend/src/jobs/scheduler.js</code> : node-cron schedules the enabled rows of <code>scheduled_jobs</code> in the time zone <code>general.timezone</code> . Each run takes a PostgreSQL advisory lock so it runs once across instances, and is recorded in <code>job_runs</code> . Schedule changes reach every instance within 30 seconds.
File engines	One PDF engine (<code>lib/pdf</code>) for every document and report, with the letterhead of the primary company of the Company master; XLSX, CSV and ZIP writers in <code>lib/</code> . No third-party document library.
PostgreSQL	System of record for business, financial, configuration, security and log data (documents 02 and 03).
File store	<code>UPLOAD_DIR</code> (<code>/app/uploads</code> in the container): uploaded documents and photos, generated reports and statements, registered in the <code>documents</code> table.
SMTP server	Office 365 (<code>smtp.office365.com</code> , port 587, STARTTLS) delivers the e-mail queued in <code>email_outbox</code> : quote approval links, broker slip requests and placement orders to insurers, password reset codes, renewal and collection reminders, report schedules and remittance statements.

2.2 Front-end application structure

The SPA (`brokerverse/src`, 848 JavaScript and JSX files) is organised by functional area:

Folder	Content
<code>agentModule/</code>	Operations screens: leads, quotation wizard, policy, endorsements, claims, renewals, collections, open items, payments, the Sales Dashboard and the customer quote approval page. The folder name is historical; there is no agent sign-in.
<code>module/</code>	Back-office areas: Administration, BankReconciliation, ClaimsModule, Commission, CorrectionJV, ExecutiveDashboard, FinanceMastersModule, GeneralMasters, Incentive, JournalVoucher, PaymentVoucher, PeriodEnd, PettyCashManagement, Placement, ProductConfigurator, Receipts, Reinsurance, Remittance, Renewal, Reports, Reversals, SystemSettings, UnderwritingModule, accounts
<code>services/</code>	43 API client services (one per backend area, for example <code>placementService.js</code> , <code>periodEndService.js</code> , <code>bankReconciliationService.js</code> , <code>importService.js</code>)
<code>routes/</code>	<code>MainRoute.js</code> (416 routes) and <code>ProtectedRoute</code> (route guard)
<code>redux/, context/, hooks/</code>	Redux Toolkit store (reset on sign-out), notification context, currency formatting hook
<code>utility/, utils/</code>	Shared helpers: HTTP interceptor and session refresh, menu permissions, dialogs and toasts, date, number and phone formatting, idle time-out, system settings, a logger that is silent in production

Folder	Content
components/	Reusable UI: layout, header, side bar, file upload, dialogs, toasts, status badges, stepper, notification drop-down
locales/, i18n.js	i18next resources: English (en.json, complete) and Thai (th.json, partial). Filipino is listed in the settings but has no translation file.

3 Deployment view

3.1 Production deployment on the existing BrokerVerse URL

`deploy/README.md` is the checklist the iorta TechNXT DevOps team follows to connect the source to the existing BrokerVerse URL; `deploy/REFERENCE.md` gives every environment variable, the seed data, the scheduled jobs, the health checks and the production start-up rules. Figure 2 shows the result. The AWS service names in dashed boxes and the "e.g." choices are options the checklist leaves open.

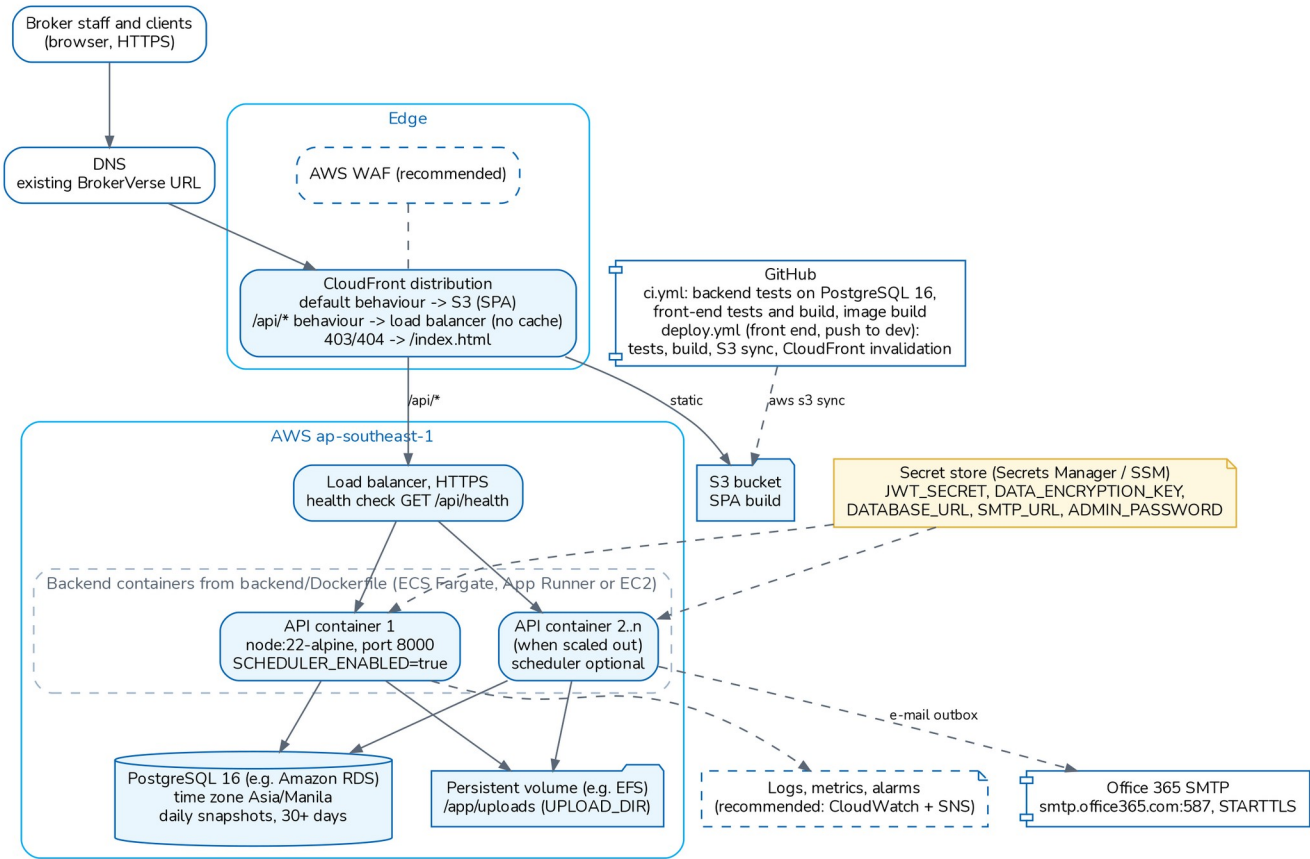


Figure 2. Production deployment on the existing BrokerVerse URL (AWS ap-southeast-1)

Layer	Fact from the baseline	To be confirmed by DevOps
Front-end build and hosting	<code>brokerverse/.github/workflows/deploy.yml</code> : on a push to dev, Node 22, <code>npm ci</code> , front-end tests, a check that the repository variable <code>REACT_APP_BASE_URL</code> is set, <code>build, aws s3 sync build/ s3://\$S3_BUCKET --delete</code> , CloudFront invalidation <code>/*</code> , region <code>ap-southeast-1</code> . A failing test stops the deployment.	GitHub OIDC role instead of long-lived access keys; S3 versioning for roll-back
API address	Same-domain option recommended: CloudFront behaviour <code>/api/*</code> to the backend load balancer, all methods and headers, no caching. Separate domain possible with <code>CORS_ORIGINS</code> .	Choice of option
CloudFront errors	Custom error responses 403 and 404 return <code>/index.html</code> with status 200, so deep links work.	
API runtime	<code>backend/Dockerfile</code> : <code>node:22-alpine</code> , <code>npm ci --omit=dev</code> , runs as user <code>node</code> , port 8000, <code>HEALTHCHECK on /api/health</code> . Behind a load balancer with HTTPS.	ECS Fargate, App Runner or EC2; number of instances
Database	PostgreSQL 16, created empty with a login that may create tables; database time zone <code>Asia/Manila</code> . The API migrates and seeds it on start. <code>?sslmode=require</code> on RDS.	Amazon RDS class, Multi-AZ, backup retention (at least 30 days in the checklist)
Files	A persistent volume at <code>UPLOAD_DIR (/app/uploads)</code> is required; a container without it loses documents on restart.	EFS or another shared volume when more than one instance runs
E-mail	Office 365: <code>SMTP_URL=smtp://connect%40iortatechnxt.com:<password>@smtp.office365.com:587</code> , sender <code>BrokerVerse <connect@iortatechnxt.com></code> ; Authenticated SMTP turned on for the mailbox. E-mail is off until "Send e-mails" is switched on in Master > Configuration.	App password if the tenant enforces multi-factor sign-in
Secrets	<code>JWT_SECRET</code> , <code>DATA_ENCRYPTION_KEY</code> , <code>ADMIN_PASSWORD</code> , <code>DATABASE_URL</code> , <code>SMTP_URL</code> in a secret store (AWS Secrets Manager, SSM Parameter Store or the GitHub environment), never in a repository. The production start refuses unsafe values.	Secrets Manager or SSM
Repository CI	<code>.github/workflows/ci.yml</code> on every push to <code>brokerverse-platform</code> or <code>main</code> and on pull requests: backend lint and tests against PostgreSQL 16, front-end tests and production build, backend image build.	Image registry and backend release pipeline

3.2 Docker Compose

`docker-compose.yml` runs the system on one server, a test system or for a trial. It has three services. `db` runs `postgres:16-alpine` with a `pg_isready` health check and the volume `db-data`. `api` is built from `backend/`, runs with `NODE_ENV=production` by default, refuses to start without `JWT_SECRET`, `DATA_ENCRYPTION_KEY`, `ADMIN_PASSWORD`, `CORS_ORIGINS` and `PUBLIC_BASE_URL`, and keeps files in the volume `uploads`. `web` is built from `brokerverse/` (nginx 1.27) and publishes port 8080; it serves the SPA and forwards `/api/` to the API container. A TLS proxy must be placed in front of port 8080.

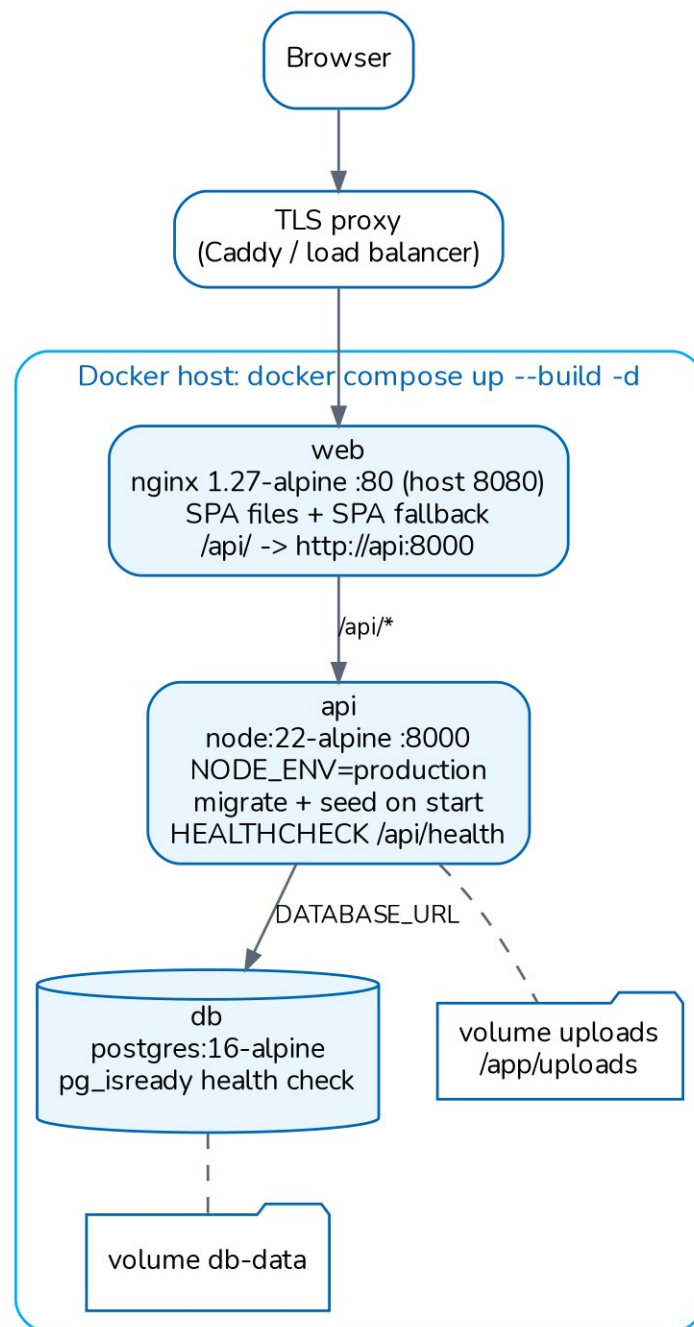


Figure 3. Docker Compose deployment (docker-compose.yml)

4 Request processing

4.1 API request pipeline

Every request passes the same middleware chain before it reaches a handler. The route registry (`backend/src/lib/registry.js`) places `requireAuth` first on every route that is not declared `auth: false`, so an endpoint cannot be left open by mistake. 14 routes are public by design: sign-in, the two-factor step, refresh, logout, forgot and reset password, the password policy, the public system settings (two routes), the version, the customer quote approval, a file download with a signed link (two routes) and a generated report download with a signed token.

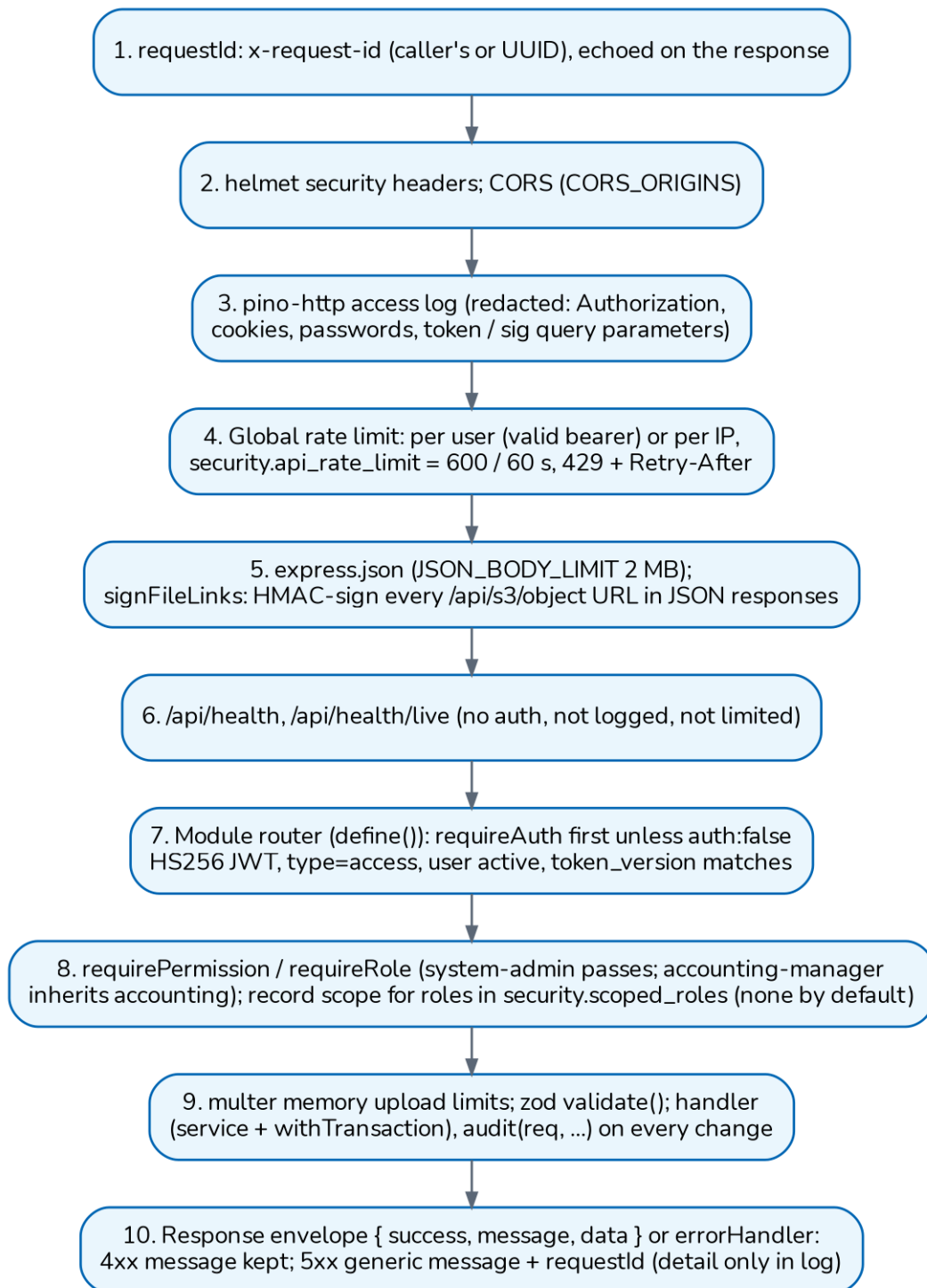


Figure 4. Request pipeline of the BrokerVerse API

4.2 Authentication and session flow

Sign-in, token refresh and signed file links

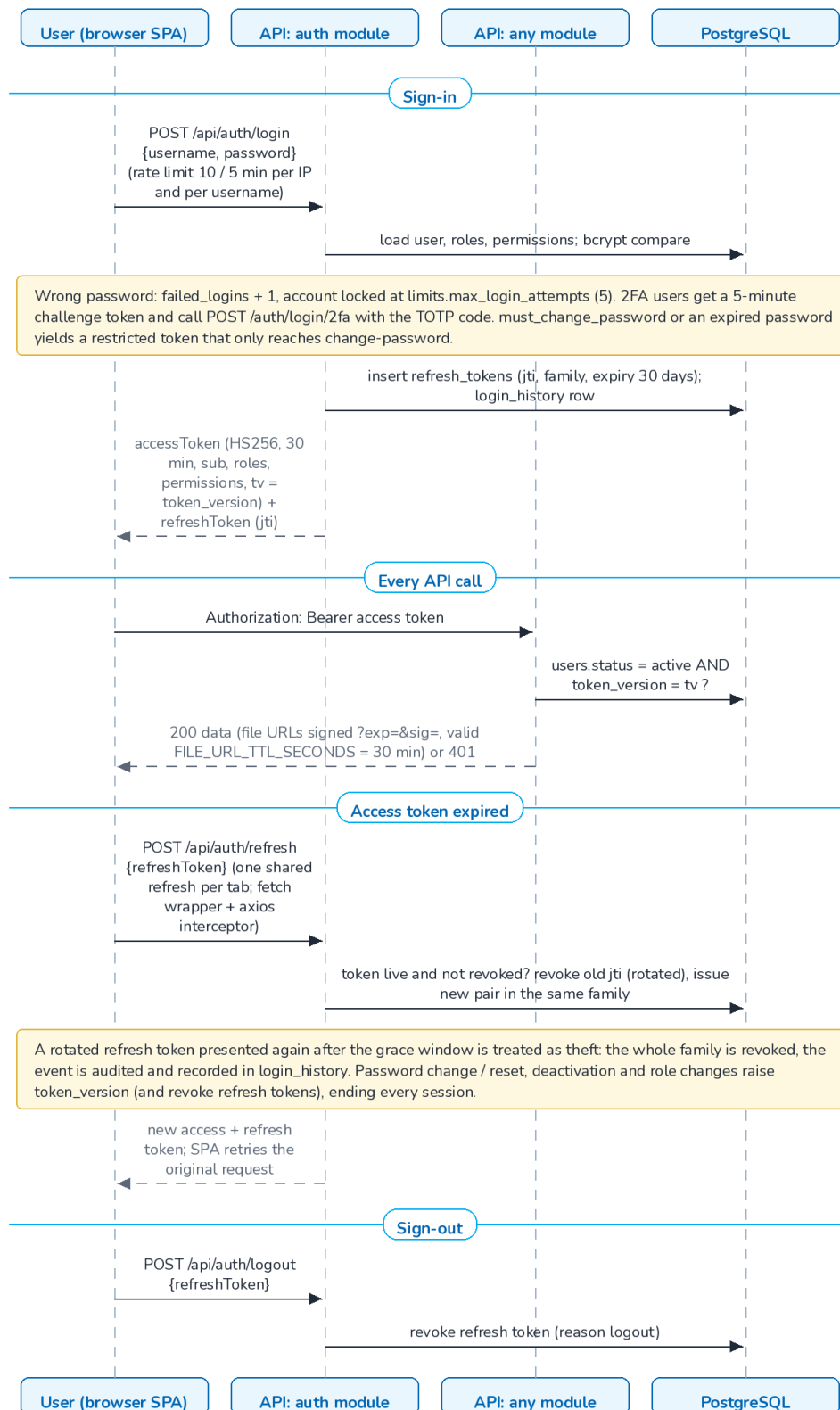


Figure 5. Sign-in, refresh-token rotation and signed file links

Mechanism	Implementation
Access token	JWT, HS256 only, lifetime <code>JWT_ACCESS_TTL_SECONDS</code> (default 1,800 s). Claims: user id, username, effective roles, permissions, <code>type=access</code> , <code>tv</code> (token version) and an optional <code>restrict</code> (<code>enrol2fa</code> , <code>pwchange</code>).
Per-request check	<code>authenticate()</code> verifies the signature and type, then reads <code>users.status</code> and <code>users.token_version</code> . An inactive user or a changed token version ends the session at once (HTTP 401).
Refresh token	JWT with a <code>jti</code> recorded in <code>refresh_tokens</code> (lifetime <code>JWT_REFRESH_TTL_SECONDS</code> , default 30 days), rotated on every use. Reuse of a rotated token after the grace window revokes the whole token family and is audited.
Revocation	Password change or reset, administrator reset, deactivation and role or permission changes raise <code>token_version</code> and revoke refresh tokens.
Sign-in protection	Limit of 10 attempts per 5 minutes per IP and per username (<code>security.login_rate_limit</code>); account locked after <code>limits.max_login_attempts</code> (5) wrong passwords; every attempt recorded in <code>login_history</code> .
Password policy	Minimum length 8, upper and lower case, digit and symbol, history of 5, maximum age 90 days (<code>security.password_*</code>). A must-change or expired password gets a restricted token. Users created by an administrator or by <code>scripts/provision-users.js</code> must change the initial password at the first sign-in.
Two-factor	TOTP (RFC 6238) in <code>lib/totp.js</code> ; secrets encrypted at rest with AES-256-GCM under <code>DATA_ENCRYPTION_KEY</code> . Roles that must use it are set in <code>security.require_2fa_roles</code> (empty by default; the checklist suggests system-admin, accounting and accounting-manager).
Front end	Tokens kept in <code>localStorage</code> . <code>utility/sessionRefresh.js</code> wraps <code>fetch</code> and <code>utility/interceptor.js</code> wraps <code>axios</code> with one shared refresh. Idle sign-out after <code>limits.session_idle_minutes</code> (30) with a one-minute warning.
File links	Stored documents are served by <code>GET /api/s3/object/<key></code> only with a bearer token or an HMAC-signed link (<code>?exp=&sig=</code>) valid for <code>FILE_URL_TTL_SECONDS</code> (1,800 s). The API signs every object URL in JSON responses on the way out.

Limitation: Tokens are stored in the browser's local storage; this is accepted for go-live (`deploy/README.md`, section 9). A change to an `httpOnly`, `SameSite` refresh cookie needs a backend change. The global and sign-in rate limits are counted per API process, so with several instances the effective limit is multiplied by the number of instances.

5 Module map

5.1 Backend modules

The API is a modular monolith. Each module owns its routes, services and tables and shares the database and the `lib/` services. Module routers are loaded in folder order (the `system` module first) and mounted under `/api`. Some modules mount extra prefixes: `placement` serves `/broker-slips` and `/placements`, `posting-rules` also serves `/account-determination`, `payments` serves `/open-items` and `/petty-cash`.

Module (backend/src/modules)	Routes	Responsibility
auth	15	Sign-in, two-factor challenge and enrolment, refresh, logout, change, forgot and reset password, profile
users	16	Users and roles (User Management), role permissions and inheritance, sessions, sign-in history
leads	11	Leads and prospects, statistics, lead report, lead upload
clients	6	Clients and customer codes (<code>/clients</code> , <code>/customers</code>)
addresses	5	Provinces, cities, districts and postal codes
quotations	38	Quotation wizard, motor tariff, premium calculation, customer approval link, submission to the insurer, conversion to policy, quotation masters, e-mail
placement	26	Broker slips to several insurers, insurer offers and comparison, placement slips, binding per co-insurer, policy issue and "Record Issued Policy"; the journey per product type (<code>placement.journey</code>)
policies	12	Policy list and detail, KYC, payment capture for accounting confirmation, policy documents, policy upload
endorsements	8	Endorsements and cancellations with re-pricing, return premium and billing
documents	5	Server-rendered PDFs (quotation, policy schedule, receipt) and upload template downloads
claims	16	Claim registration with acceptance check, lifecycle, settlement with maker-checker, settlement cash from insurers to the claimant, documents and letters, claims reports
renewals	42	Renewal pipeline, renewal quotes, notices, batches and queue, workspace, analytics, win-back campaigns
receipts	16	Official receipts, application to bills, billing statements, receivables, open items import for go-live
collections	9	Collection items, follow-up actions, due-date reminders, KPIs
accounting	27	Journal engine, ledgers per client and policy, open-item matching and write-off, trial balance, chart of accounts and its upload
posting-rules	14	Posting rules per business event with versions and simulation; Account Determination (the GL account of each account role)
journal-vouchers	8	Journal vouchers with maker-checker, correction and reversal
period-end	45	Fiscal years, period status, month-end and year-end close, recurring and accrual journals, financial statements, tax codes, BIR Form 2307, go-live opening balances
bank-reconciliation	38	Statement formats and import, matching rules and workspace, adjustments, stale cheques, monthly reconciliation with approval
disbursements	15	Payment vouchers, invoice lists, cheques, commission and insurer payouts
payments	29	Open items, payments and petty cash (funds, requests, disbursements, receipts, replenishment)

Module (backend/src/modules)	Routes	Responsibility
commission	19	Commission accrual, eligibility, approval, payout and clawback; referrer accounts and dashboard
commission-rates	7	Commission rate matrix per insurer, product, line and policy type with effective dates
remittance	86	Remittances to insurers, bills, settlements, multi-level approvals and delegations, statements, refunds due from insurers, direct bill (debit notes, collections)
reinsurance	38	Reinsurers, treaties, cessions, recoveries, bordereaux, reconciliation, analytics
incentive	22	Incentive programmes, calculation batches with maker-checker, results, reports
product-configurator	72	Product templates, components, rating, rules, risk mapping, versions
masters	13	Generic master data screens driven by <code>master_types</code> , master uploads
document-numbering	5	Document number series: prefix, pattern, width, reset rule and next number
reports	13	Report catalogue, on-screen run, file generation (CSV, XLSX, PDF), schedules
dashboard, search	6	Executive, sales, processing and claims dashboards; global search
notifications	8	In-app notifications and unread count
uploads	11	File upload, signed download, existence check, delete (<code>/s3</code> , <code>/upload</code>)
settings, system-settings	11	Configuration keys by group, audit trail query; branding and system settings (public read for the sign-in page)
schedules	4	Scheduled jobs: list, run history, edit, run now
system	1	<code>GET /api/version</code> : build and runtime information

Route counts come from the route registry through `collectRoutes()` in `backend/src/tools/export-api.js`, the same list that `npm run export:api` writes to `backend/docs/api/openapi.json`: 717 operations (331 GET, 277 POST, 74 PUT, 28 DELETE, 7 PATCH).

5.2 Front-end areas and backend modules

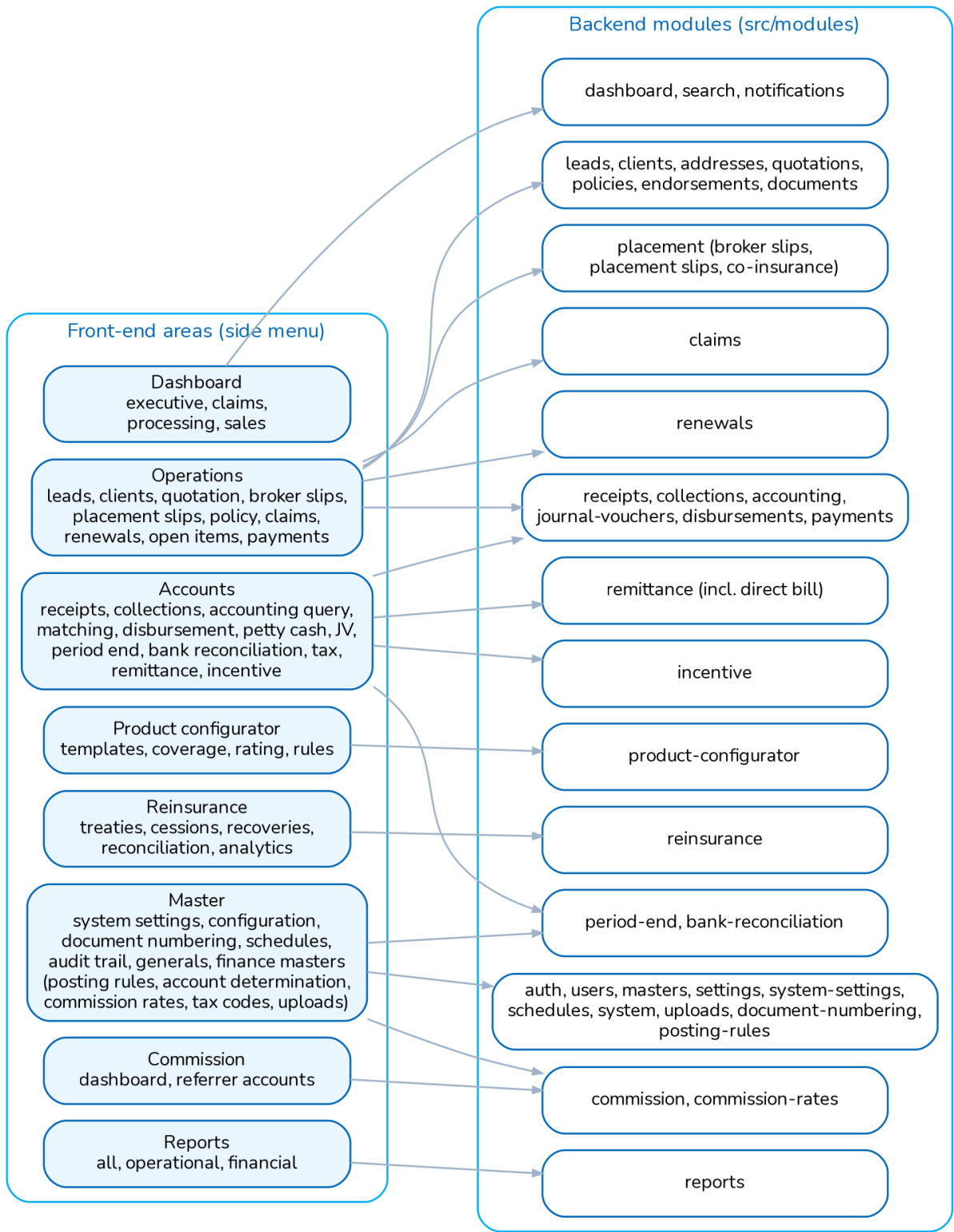


Figure 6. Front-end areas (menus) and the backend modules they use

6 Integration points

Integration	Direction	Description
SMTP e-mail	Outbound	<code>lib/mailler.js</code> queues messages in <code>email_outbox</code> . The <code>email-outbox</code> job (every 5 minutes) sends up to 50 queued messages per run through <code>nodemailer</code> and <code>SMTP_URL</code> when <code>notification.email_enabled</code> is true; up to 5 attempts, then <code>failed</code> . Sender: <code>notification.from_address</code> (BrokerVerse <connect@iortatechnxt.com>; Office 365 refuses any other sender for that mailbox).
Client quote approval	Inbound (public)	The approval e-mail carries a signed token (<code>quotations.approval_link_ttl_hours</code> , 168 h). <code>POST /api/quotations/approve-by-customer</code> previews or accepts; only the latest token of a quotation is valid.
Insurers	Outbound (e-mail, files)	Broker slip requests for quotation, placement orders to the lead insurer and co-insurers, remittance statements and bills, commission debit notes; PDF, CSV and XLSX files. Insurer answers are keyed as offers. There is no insurer API integration.
Reinsurers	Outbound (files)	Bordereaux and reconciliation files generated as CSV.
Uploads and go-live imports	Inbound (files)	Masters, chart of accounts, leads, quotations, policies, receipts, payment vouchers, opening balances, open items, bank statements and remittance bulk files through <code>importUpload()</code> : one file of at most <code>IMPORT_MAX_MB</code> (10 MB), decompressed size capped at <code>IMPORT_MAX_INFLATED_MB</code> (50 MB) and <code>IMPORT_MAX_ROWS</code> (20,000) rows. The 39 templates in <code>docs/templates</code> are generated from the importers' own column lists and checked by <code>backend/test/upload-templates.test.js</code> .
Bank statements	Inbound (files)	CSV or XLSX statements read with a statement format (layouts of three Philippine banks and a generic layout ship as standard) for bank reconciliation.
User provisioning	Operations	<code>scripts/provision-users.js</code> creates named users from a CSV kept outside the repository (dry run unless <code>CONFIRM_PROVISION=yes</code>).
Payment gateway	Not integrated	Policy payments are captured by staff with proof of payment (<code>policy_payments</code>) and confirmed by Accounting.
API documentation	Developer	<code>npm run export:api</code> writes <code>backend/docs/api/openapi.json</code> , a Postman collection and environment, and <code>BrokerVerse_API_Touchpoints.xlsx</code> and <code>.csv</code> from the route registry.
Monitoring	Operations	<code>GET /api/health</code> (readiness), <code>GET /api/health/live</code> (liveness), <code>GET /api/version</code> (build, commit from <code>GIT_COMMIT</code> , database reachability, pending migrations); JSON logs on stdout (document 11).

7 Security architecture summary

7.1 Roles and permissions

Permissions are codes `read:<module>` and `write:<module>` for 27 modules, plus `approve:period-end` and `approve:bank-reconciliation`: 56 permission rows. Seven roles are seeded (`backend/src/db/seed.js` and migration `0140_broker_roles.sql`). A role can inherit another role (`roles.inherits`); the effective roles of a user are resolved by the database function `user_effective_roles()`. Administrators can create further roles and change grants in User Management.

Role	Access (seeded grants)
System Administrator (Super Admin Access), <code>system-admin</code>	Every permission; configuration, users and roles. Only a System Administrator can grant this role or change an administrator account, and nobody can change their own roles or status.
Sales & Marketing (Account Executive), <code>sales</code>	Leads, clients, quotations, policies, endorsements, renewals, reports, notifications; read products, masters and claims
Processing Team (Placement & Policy Processing), <code>processing</code>	Broker slips and placement (quotation permissions), policies, endorsements, renewals, reinsurance, product templates, clients, reports; read leads, masters and claims
Operations (Client Servicing), <code>operations</code>	Leads, clients, quotations, policies, endorsements, renewals, reports; read claims, masters and products
Claims, <code>claims</code>	Claims, reports; read clients, policies and masters
Accounting, <code>accounting</code>	Receipts, collections, disbursements, commission, remittance, incentive, journal vouchers, period end, bank reconciliation, reports; read clients, policies, claims, masters and schedules
Accounting Manager, <code>accounting-manager</code>	Inherits Accounting, plus <code>approve:period-end</code> and <code>approve:bank-reconciliation</code> : approves month-end and year-end closes and bank reconciliations, posts into soft-closed periods and reopens periods

There is no agent sign-in. Referrers and sub-agents are a master (`commission_referrers`): Sales & Marketing enter their business and Accounting pays them. The server enforces permissions on every route (`requirePermission`, `requireRole`); the front end also hides menus and blocks routes by role (`utils/menuPermissions.js`, deny by default, and `routes/ProtectedRoute`). Receipts are an Accounting function (segregation of duties).

7.2 Controls

Control	Implementation
Record scope	Users whose roles are all listed in <code>security.scoped_roles</code> see and act only on records they own or that belong to their clients (<code>lib/scope.js</code>); another user's record answers 404. The setting is empty in the seed, so no role is scoped.
Maker-checker	A different user must approve: journal vouchers, payment vouchers and cheques, commission payouts, remittances and settlements (multi-level with delegation), commission debit notes, claim settlements, incentive calculations, reinsurance treaties, petty cash requests, month-end and year-end closes, bank reconciliations, quotation and renewal approvals (<code>lib/makerChecker.js</code> , switch <code>finance.maker_checker_enabled</code>).
Period control	Every journal passes <code>assertPostingAllowed</code> : open periods accept all postings, soft-closed periods only postings by <code>approve:period-end</code> , closed and locked periods none. A trigger on <code>journal_vouchers</code> checks balance and period again at posting.
Audit	<code>lib/audit.js</code> writes <code>audit_log</code> (user, entity, id, action, before and after JSON, IP) from every mutating handler; claims keep a field-level trail (<code>claim_field_changes</code>); period and reconciliation status changes have their own history tables; sign-ins are in <code>login_history</code> .

Control	Implementation
Input validation	zod schemas on request bodies and parameters where declared; parameterised SQL only, with dynamic identifiers taken from white lists.
Uploads	Type checked against <code>uploads.allowed_types</code> by file signature and the detected type stored; per-file size <code>UPLOAD_MAX_MB</code> (10 MB) and <code>UPLOAD_MAX_FILES</code> (10); files served with <code>nosniff</code> and a sandbox CSP, and anything other than images and PDF as a download.
Transport and headers	TLS at the load balancer and CloudFront; helmet headers on the API; nginx adds <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> and <code>Referrer-Policy</code> in the Compose option; CORS limited to <code>CORS_ORIGINS</code> (* refused in production).
Secrets	Production start refuses a missing, placeholder or short (under 32 characters) <code>JWT_SECRET</code> or <code>DATA_ENCRYPTION_KEY</code> , equal keys, <code>CORS_ORIGINS=*</code> or a localhost <code>PUBLIC_BASE_URL</code> . Reset codes are stored only as HMAC hashes.
Logging	pino JSON logs with redaction of authorization headers, cookies, passwords, refresh tokens and token or signature query parameters; 5xx responses return a generic message with the request id.

8 Key business flows

8.1 Lead, placement, policy and billing

Lead to placement to policy to billing

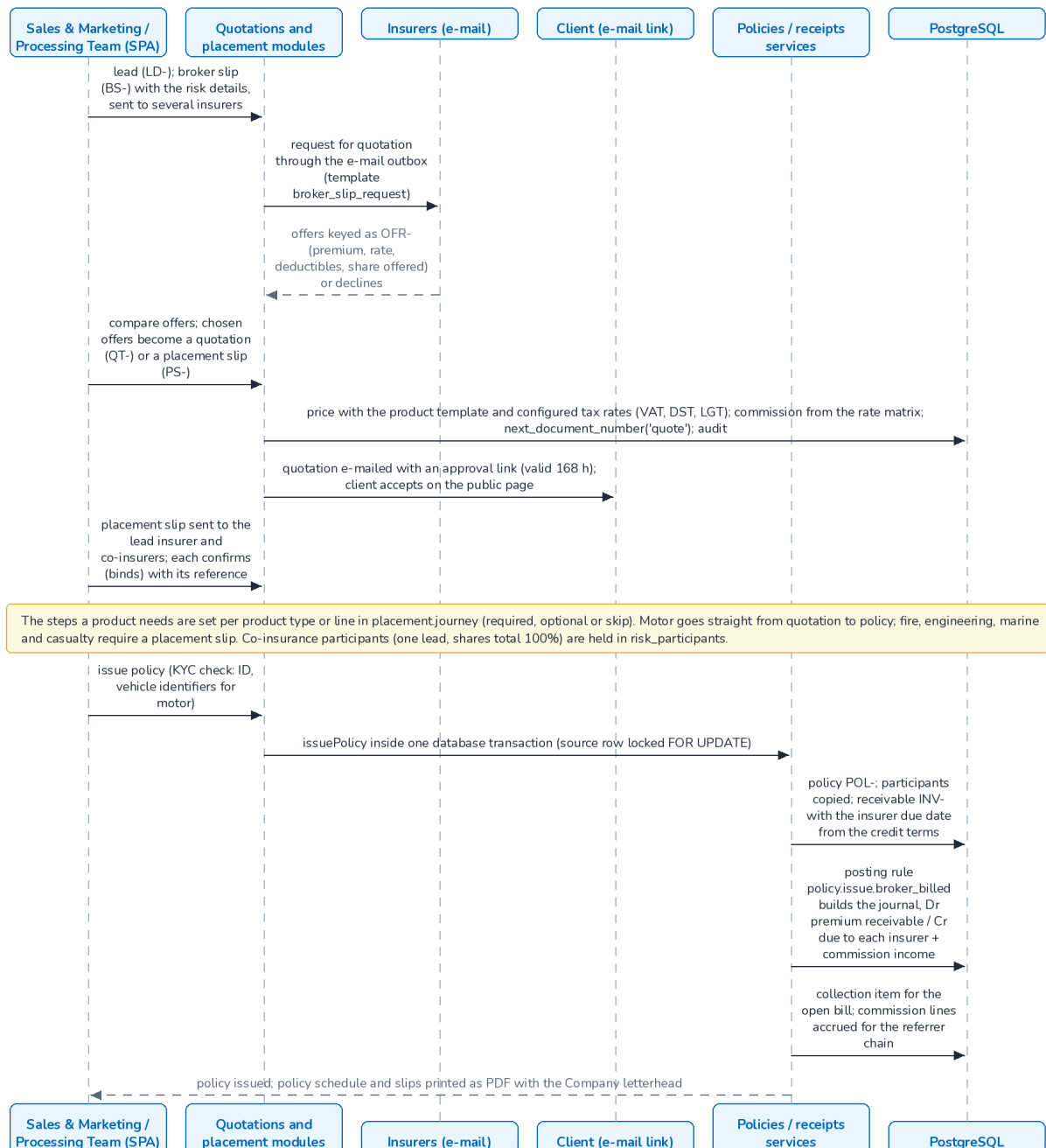


Figure 7. Lead to placement to policy to billing

8.2 Receipt, commission and remittance

Receipt, commission payout and remittance to the insurer

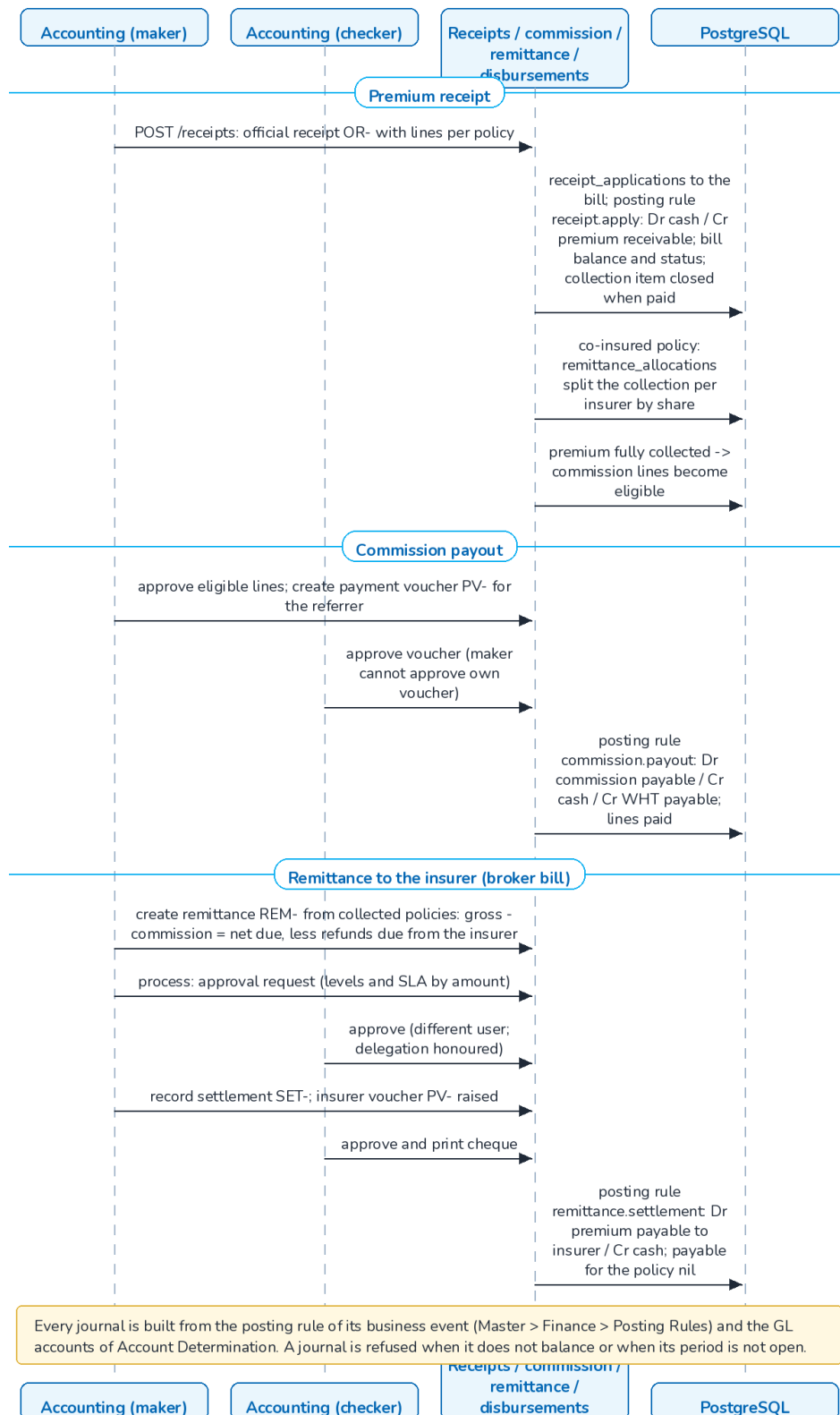


Figure 8. Receipt, commission payout and remittance to the insurer

8.3 Direct bill

Direct bill: commission debit note to the insurer

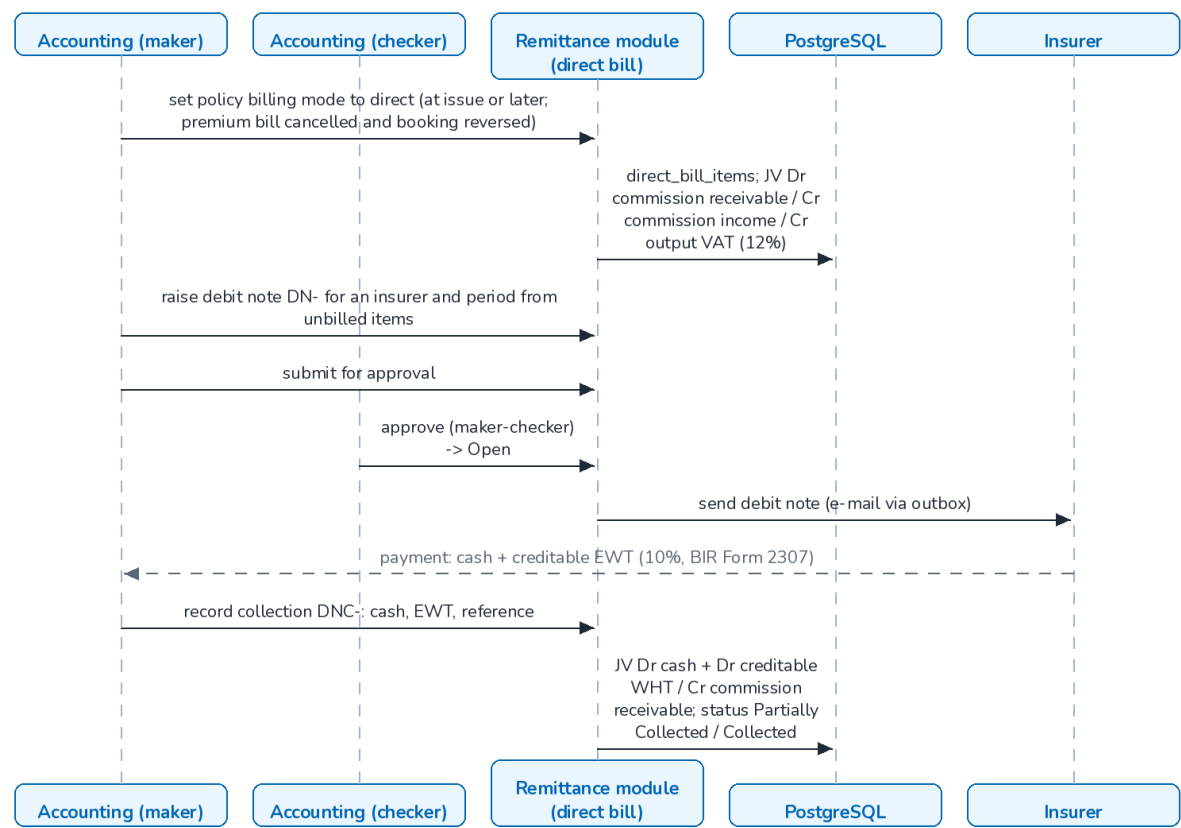


Figure 9. Direct bill: commission debit note to the insurer

8.4 Claims

Claim registration to settlement and cash

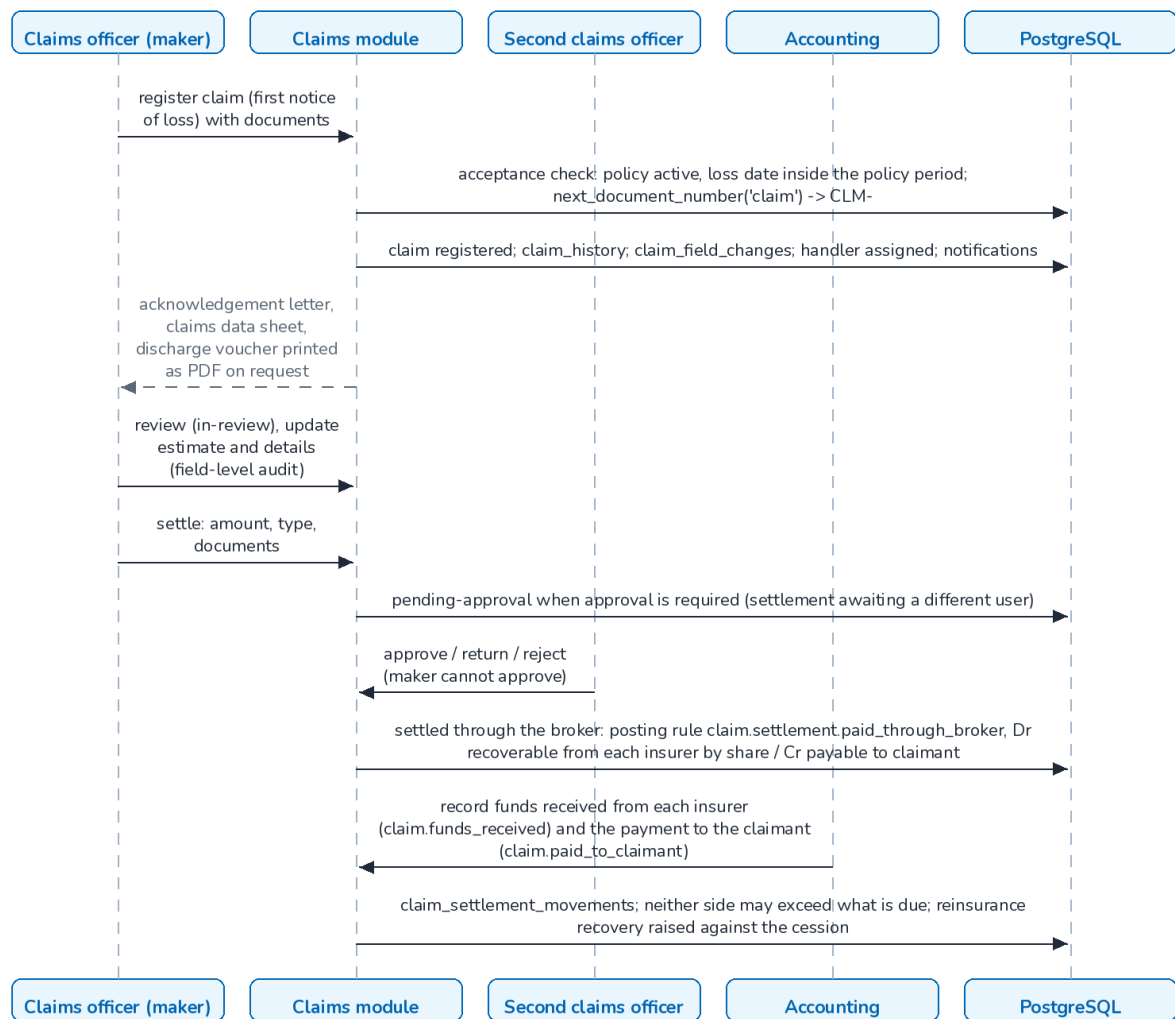


Figure 10. Claim registration to settlement and cash

8.5 Renewal

Renewal pipeline to renewed policy

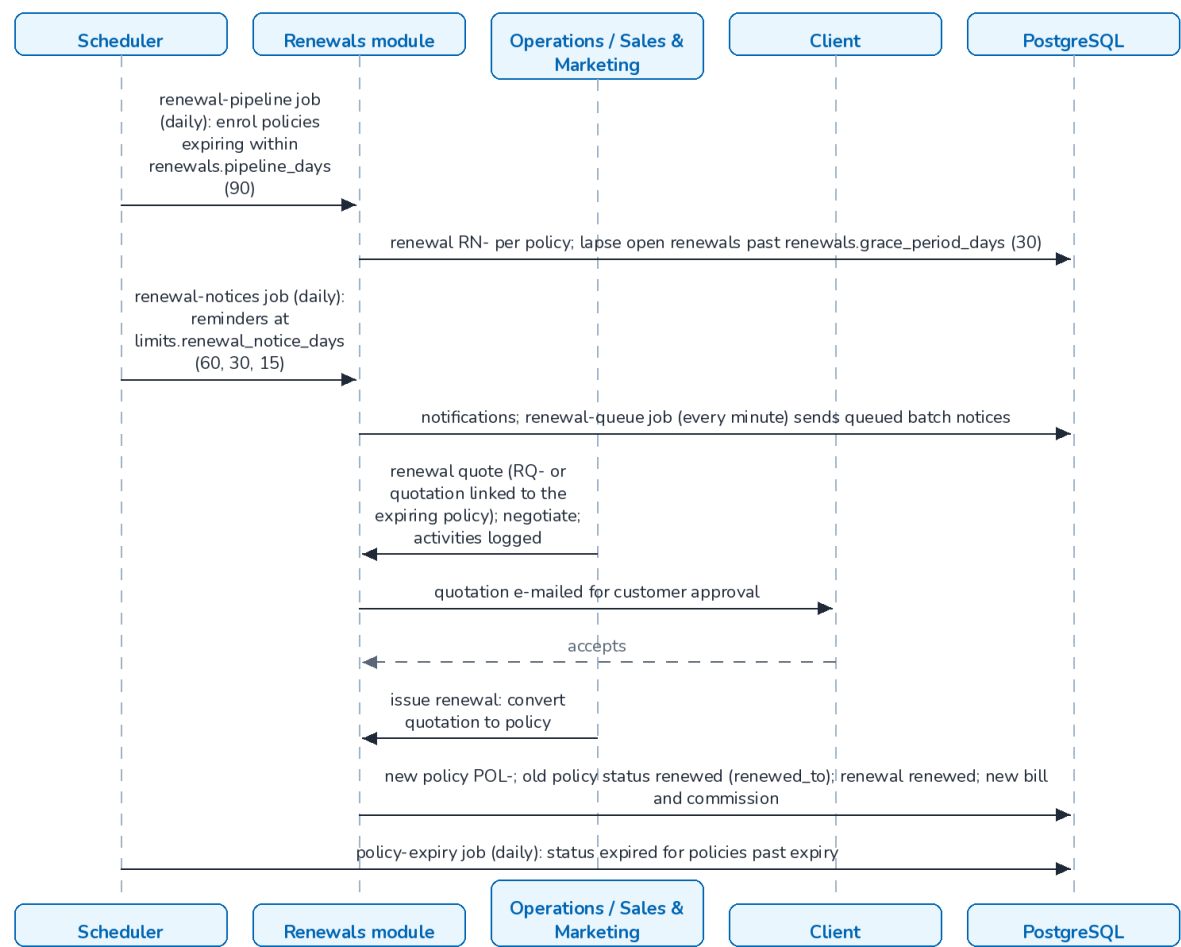


Figure 11. Renewal pipeline to renewed policy

8.6 Period end

Accounting prepares the month-end close for a period (run MEC-): accrual and recurring journals, unearned commission deferral, FX revaluation and the checklist. The manual checklist items are signed and the run is submitted. When `accounting.period_close_requires_approval` is on, an Accounting Manager approves and the period becomes closed. Executing a run again first reverses its own earlier journals, so a rerun gives the same ledger. The year-end close (run YEC-) needs twelve closed periods; it posts the closing entries in period 13, carries the balance-sheet balances into `opening_balances` of the next year, locks the year and creates the next one. At go-live, the old system's trial balance is loaded into `opening_balances` and the open premium receivables as open items, with the templates in `docs/templates`.

8.7 Scheduled jobs

All cron expressions are read in the business time zone (`general.timezone`, Asia/Manila), whatever the time zone of the server clock. The seed and the migrations create 15 jobs; 10 are enabled by default.

Job (<code>scheduled_jobs.code</code>)	Cron	Enabled	Work
policy-expiry	15 0 * * *	yes	Mark policies past their expiry date as expired
quote-expiry	30 0 * * *	yes	Expire quotations older than <code>limits.quote_validity_days</code>

Job (scheduled_jobs.code)	Cron	Enabled	Work
housekeeping	45 2 * * *	yes	Delete operational rows past their retention period (System Settings > Housekeeping; the audit trail is kept)
daily-reports	0 5 * * *	yes	Generate the daily reports; purge generated reports older than <code>reports.retention_days</code> (90)
renewal-pipeline	30 5 * * *	yes	Enrol policies expiring within the pipeline window; lapse renewals past the grace period
renewal-notices	0 6 * * *	yes	Renewal reminders at the configured notice days
receivable-ageing	0 7 * * *	yes	Recompute the age and ageing bucket of open bills
collection-reminders	0 8 * * *	yes	E-mail and notify clients with premiums falling due
email-outbox	* / 5 * * * *	yes	Deliver queued e-mails
renewal-queue	* * * * *	yes	Send queued renewal batch notices, retry stale queue items
accrual-reversal	30 0 1 * *	no	Reverse last month's accrual journals on day 1
recurring-journals	15 1 * * *	no	Create the journals of recurring templates that are due
period-auto-soft-close	0 2 * * *	no	Soft-close periods after the grace days
bank-auto-match	45 5 * * *	no	Run the automatic bank matching rules
month-end-reminder	0 8 * * *	no	Remind Accounting of the coming month-end

Report schedules created under Reports > Schedules add further jobs (`report-<id>`, handler `scheduledReport`). The period-end and bank-matching jobs are disabled by default; Accounting switches them on in Master > Schedules when the business is ready.

9 Architecture decisions and constraints

Decision	Rationale and consequence
Modular monolith	One deployable API keeps an operation (policy, bill, journal, commission) in one database transaction. Modules can be split later along the folder boundaries.
Stateless API	Sessions are tokens; state is in PostgreSQL and the file store, so instances can be added behind a load balancer (document 07). The settings cache is per instance but checks the table version every 5 seconds, so a change reaches every instance within seconds. Rate-limit counters stay per instance.
Migrations and seed on start	Every instance takes an advisory lock, applies pending migrations (each in its own transaction) and the idempotent seed before it reports ready. Instances that start together wait for the first one. A failed migration blocks start-up (documents 07 and 09).
Posting rules as data	Every system journal is built from the posting rule of its business event and the account roles of Account Determination. Accounting can change the GL treatment without a code change; each rule version has an effective date.
Configuration over code	351 settings in <code>app_settings</code> (tax rates, account roles, limits, templates, security policy, retention periods), 52 document number series and the commission rate matrix are edited in the application.
One PDF engine, no third-party document libraries	PDF, XLSX, CSV and ZIP writers are part of the code base. Every PDF carries the letterhead of the primary company of the Company master, so the broker's own details print on every document and report once the master is updated.
Business time zone	Business dates, document number years, column defaults and job schedules use <code>general.timezone</code> (Asia/Manila); the database time zone is also set to Asia/Manila. Instants are stored in <code>timestampptz</code> .
Local file store behind an S3-style API	<code>uploads/storage.js</code> keeps an S3-like key space on disk. With more than one instance the volume must be shared (for example EFS) so every instance sees the same files.

10 Related documents

This document is one of eleven that together form the BrokerVerse OOTB Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture (this document)	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

11 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Account determination	Settings accounting.account.<role> that give the GL account of each account role (cash in bank, premium receivable, due to insurer, commission income ...) used by the posting rules.
Advisory lock	PostgreSQL application-level lock (pg_try_advisory_lock, pg_advisory_lock) used so a job runs on one API instance at a time and only one instance applies migrations.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
Audit log / audit trail	Table audit_log: who changed which record, when, from which IP, with the before and after values.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
BIR	Bureau of Internal Revenue (Philippines): tax authority whose rules drive record keeping and withholding tax certificates.
BIR Form 2307	Certificate of creditable tax withheld at source, issued by the payor to the payee each quarter.
Broker slip	Request for quotation (BS-) that the broker sends to several insurers for one client risk; the answers are recorded as insurer offers.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
Company master	Master record of the broker company; the primary company is the letterhead printed on every document and report.
CORS	Cross-Origin Resource Sharing: browser rule set that the API restricts to the configured web origins (CORS_ORIGINS).
CRA / craco	Create React App (react-scripts) build tool chain, customised with craco (Create React App Configuration Override).
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
CSP	Content Security Policy: HTTP header restricting what a page may load or run.
Debit note (DN)	Commission debit note raised by the broker to an insurer for direct-billed policies.
Direct bill	Billing mode in which the insurer collects the premium and the broker bills the insurer for its commission.
ECS / Fargate	Elastic Container Service (AWS) and its serverless compute mode for running the API containers.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
HMAC	Keyed-hash message authentication code; signs file links and hashes one-time codes.
Housekeeping	Daily job that deletes operational rows (job runs, sent e-mails, sign-in history, expired tokens, read notifications) after the retention days in System Settings.
HS256	HMAC-SHA256 JSON Web Token signature algorithm, the only one the API accepts.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
KYC	Know your customer: identification data required before a policy is issued.
Maker-checker	Control in which a record created or submitted by one user must be approved by a different user.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Month-end close	Run (MEC-) that posts the accruals, recurring journals and valuations of a period, checks the checklist and, once approved, closes the period.

Term	Meaning
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
nginx	Web server used in the web container to serve the SPA and forward /api to the API container.
Office 365	Microsoft 365 mail service; BrokerVerse sends e-mail through smtp.office365.com with the mailbox in SMTP_URL.
Posting rule	Versioned definition of the journal lines for one business event (policy issued, receipt applied ...); every system journal is built from it.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Record scope	Rule that users holding only roles listed in security.scoped_roles see only their own records (lib/scope.js); no role is scoped by default.
Refresh token	Long-lived token (30 days) exchanged for a new access token; rotated on every use.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
SPA	Single-page application: the React front end served as static files.
TLS	Transport Layer Security (HTTPS).
Token version	users.token_version, carried in every access token; raising it ends all sessions of the user.
TOTP / 2FA	Time-based one-time password (RFC 6238) used for two-factor authentication.
Year-end close	Run (YEC-) that posts the closing entries in period 13, carries the balances into the next year and locks the year.
zod	TypeScript-first schema validation library used to validate request bodies and parameters.